

SOUTENANCE — PHASE 3

# HealthSync

API de gestion de rendez-vous médicaux — architecture REST & gRPC

Hono.js

TypeScript

PostgreSQL

gRPC

Docker

Vitest

# Au programme

01

## Contexte & périmètre

Ce que HealthBridge demandait, ce qui est livré

02

## Architecture

Deux services, REST en façade et gRPC en interne

03

## Démonstration technique

Parcours patient, documentation, sécurité en direct

04

## Code review

Trois fichiers choisis pour ce qu'ils démontrent

05

## Qualité & tests

124 tests automatisés, couverture, intégration continue

06

## Sécurité & bilan

Audit OWASP, correctifs, risques assumés

## Le MVP demandé — et ce qui est livré

### Fonctionnalités du MVP

- ✓ **Gestion des patients**      CRUD complet, recherche par nom et email
- ✓ **Gestion des médecins**      CRUD administrateur, créneaux disponibles
- ✓ **Rendez-vous**      Prise, conflits de créneaux, annulation, historique
- ✓ **Authentification**      JWT, trois rôles, endpoints protégés par rôle
- ✓ **Communication inter-services**      Service gRPC séparé, échange démontré

6

modules métier

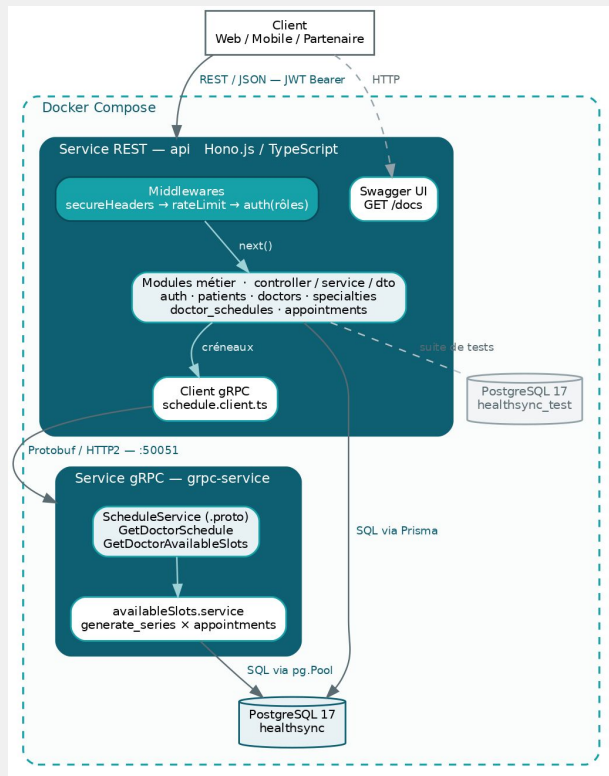
31

endpoints REST

2

RPC gRPC

## Deux services : REST en façade, gRPC en interne



### API REST — Hono.js

Point d'entrée unique des clients. Chaîne de middlewares : en-têtes de sécurité, limitation de débit, authentification et contrôle de rôle.

### Service gRPC — ScheduleService

Calcul des créneaux disponibles par croisement planning × rendez-vous, en une requête SQL avec `generate_series`. Contrat strict via fichier `.proto`.

### PostgreSQL + Prisma

Requêtes paramétrées par construction, types TypeScript générés depuis le schéma. Base de test isolée pour la suite automatisée.

## Ce que je vais vous montrer

- 1 Suite complète lancée en arrière-plan** `make test-all` — on y revient à la fin
- 2 Parcours patient de bout en bout** `make test-e2e` — inscription, réservation, conflit, annulation, historique
- 3 Documentation d'API vivante** Swagger UI sur `/docs`, servie par l'application elle-même
- 4 Protection contre la force brute** Six appels sur `/auth/login`, le sixième bascule en 429
- 5 Intégration continue et couverture** Pipeline GitHub Actions au vert, puis le rapport de couverture

## Trois fichiers, trois démonstrations

### `rateLimit.middleware.ts`

#### Middleware maison, fenêtre glissante

Stratégie choisie et documentée plutôt que subie : on ne garde que les horodatages tombant dans la fenêtre, ce qui interdit la rafale à cheval sur une remise à zéro. Rejet en 429 avant le contrôleur, donc sans requête en base.

### `availableSlots.service.ts`

#### Calcul de disponibilité côté gRPC

Une seule requête SQL avec `generate_series` et jointure externe sur les rendez-vous. Tous les paramètres passent par des placeholders — vérifié pendant l'audit OWASP.

### `parcours-rendez-vous.step.ts`

#### Test E2E en Gherkin

Le scénario est écrit en français et lisible par un non-technicien ; chaque phrase est reliée à du code qui l'exécute réellement contre un vrai serveur HTTP.

## 124 tests automatisés, exécutés à chaque pull request

107

tests Vitest

17

étapes Cucumber

84,63 %

couverture api

100 %

couverture gRPC

### Tests unitaires

Logique métier isolée, accès base simulé — validation de durée, détection de conflit, construction de filtres de recherche

### Tests d'intégration

Vraie base, vraies requêtes HTTP, chaîne de middlewares complète sur les six modules

### Tests end-to-end

Parcours patient complet en Gherkin, contre un serveur HTTP réellement démarré

## Audit OWASP Top 10 — les dix catégories passées au crible

5

déjà couvertes

4

corrigées pendant l'audit

2

risques assumés

### Les quatre correctifs apportés

#### A01 Contrôle d'accès

La liste des rendez-vous d'un médecin était accessible à tout utilisateur authentifié, exposant les identifiants d'autres patients. Restreinte aux rôles médecin et administrateur.

#### A02 Données sensibles

Le hash du mot de passe fuitait dans les réponses du module médecins. Retiré, ainsi qu'une dépendance cryptographique inutilisée.

#### A04 Conception

Aucune limite de tentatives sur la connexion. Ajout d'un rate limiting en fenêtre glissante, cinq requêtes par quart d'heure et par adresse IP.

#### A05 Configuration

Aucun en-tête HTTP de sécurité émis. Ajout de la CSP, de HSTS, de X-Frame-Options et de nosniff.

Deux de ces failles ont été trouvées en relisant le code pour rédiger la documentation d'API, pas par un outil automatique



## Ce qui reste ouvert — et pourquoi c'est assumé

### Compteur de limitation en mémoire

Remis à zéro au redémarrage, non partagé entre instances. En production : un store partagé type Redis.

### Pas de journalisation de sécurité

Une intrusion bloquée ne laisse aucune trace exploitable. En production : logs structurés et corrélation entre services.

### Dépendance vulnérable non corrigée

Faible modérée sur @hono/node-server, mais la fonctionnalité concernée n'est jamais appelée. Correctif = montée de version majeure.

### Communication en HTTP

Acceptable en développement, bloquant en production : TLS obligatoire et hébergement certifié HDS pour des données de santé.

*L'objectif du rapport n'était pas d'afficher un score parfait, mais de rendre compte honnêtement de l'état réel du code.*

# HealthSync — MVP livré

Une API de gestion de rendez-vous médicaux, du contrat gRPC à l'audit de sécurité

## Fonctionnel

Patients, médecins, rendez-vous, conflits, historique

## Authentification

JWT, refresh rotatif, trois rôles

## Inter-services

gRPC, contrat .proto, contract test

## Tests

124 tests, 84,63 % de couverture, CI

## Documentation

Swagger vivant, ADR, README, schéma

## Sécurité

OWASP Top 10 audité, quatre correctifs

[github.com/herve-beziat/HealthSync](https://github.com/herve-beziat/HealthSync)